Travail Pratique 2

Ce travail a pour objectif de réaliser une simulation d'une ville et de ses habitants. Ces habitants devront utiliser une machine à état visant à réguler leurs signes vitaux et réagir aux événements.

Valeur de la note finale	Contexte de réalisation	Durée	Usage de l'IAg
20%	En équipe de 2	2 semaines	Niveau 1

Travail à effectuer

1 Explorer le projet de départ

Importe le projet de départ fourni dans Unity et ouvre la scène nommée `City`. Cette scène contient déjà plusieurs objets servant de point de départ à ton jeu. Entre autres :

- **VirtualCamera** : Caméra du jeu utilisant *Cinemachine*. Utilise WASD pour la déplacer, QE pour la tourner et la molette de la souris le zoom. Elle est déjà fonctionnelle : tu n'as rien à toucher.
- **GameController** : Composants globaux. Contient un script répertoriant tous les acteurs importants de la scène, comme les personnages et les lieux (tu en auras besoin pour tes propres scripts). Il est accessible de partout via la classe `Finder`.
- **Characters** : Personnages du jeu. Chaque personnage est un *prefab* différent, et ils héritent tous du même *prefab* nommé `BaseCharacter`. Lorsque tu auras à ajouter un script aux personnages, il est impératif de le faire sur ce *prefab* au lieu de le faire sur chaque personnage individuellement.
- **Canvas** : Interface du jeu. Servira à afficher des informations sur les personnages. Les éléments sont déjà placés et configurés : il ne reste qu'à mettre les données à l'intérieur.

Démarre le jeu une première fois et essaie le déplacement de la caméra pour t'y habituer. En même temps, remarque les différents types de bâtiments. La couleur du sol et l'icône sur le dessus en indiquent le type. Tous ces bâtiments sont répertoriés dans le `GameController` pour les trouver facilement.

Nom et icône	Description
Maison (House)	Chaque personnage possède une maison où il va dormir. En vert.
Travail (House)	Chaque personne possède un lieu où il va travailler. En bleu.
Épicerie (Food)	Lieu où les personnages peuvent aller manger. En jaune.
Café (Social)	Lieu où les personnages peuvent aller socialiser. En rose.

Ouvre le projet C# à partir de Unity. Ce projet contient plusieurs scripts : certains déjà fonctionnels, d'autres à compléter. Bien entendu, tu auras aussi à en créer. Voici les plus importants :

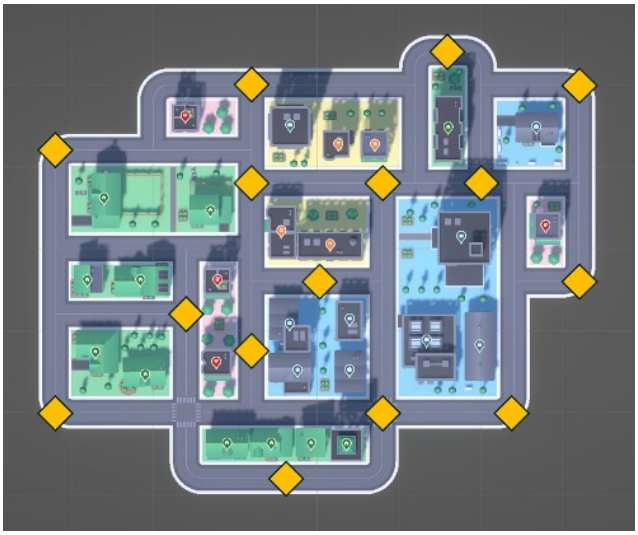
- **GameController** : Contrôleur du jeu. Tu trouveras à l'intérieur un objet `CityObjects`, contenant des références vers tous les acteurs et lieux du jeu. Cela te sera très si un personnage doit se trouver une destination spéciale (ils connaissent déjà leur domicile et leur lieu de travail).
- **Character** : Personnage pouvant se déplacer (`NavigateTo`), ramasser un déchet (`PickupTrash`), lancer un déchet (`ThrowTrash`) et saluer un ami (`GreetCharacter`). Il peut également devenir visible ou invisible pour simuler son entrée dans un bâtiment. Certains aspects sont pris en charge dans d'autres classes, tel que `CharacterVitals` pour les signes vitaux (manger, socialiser et dormir).
- **CharacterBlackboard** : Mémoire de travail d'un personnage. Conserve toutes les informations qu'il possède, comme son lieu de travail ou le dernier ami aperçu. De nombreux scripts (tel la machine à état) modifient et consultent ces données pour mener à bien leurs tâches.
- **Building** : Bâtiment dans lequel un personnage peut se rendre pour satisfaire un besoin. Pour simuler leur entrée dans un bâtiment, les personnages doivent devenir invisibles. Pour rappel, tous les bâtiments sont listés dans le `GameController`.
- **Trash** : Déchet lancé par un personnage *malpropre*. Les personnages *propres* doivent les ramasser. Le type de personnage est indiqué dans la classe `Character`. Chaque personnage décide d'un moment pour lancer un déchet (délai entre 5 et 60 secondes, choisi aléatoirement).
- **IDestination** : Position vers laquelle un personnage peut se déplacer. Les bâtiments, les déchets et même les personnages sont considérés comme des destinations. Une destination peut devenir inaccessible (un déchet ayant été ramassé n'est plus accessible).

Attention!

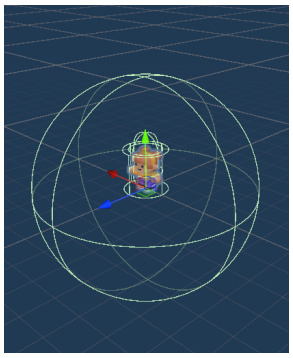
Avant de programmer, assures-toi de bien comprendre le code de départ : tu ne voudrais travailler en double et refaire ce qui existe déjà. Afin d'identifier les sections importantes, certains bouts de code sont cachés dans des régions. S'il est écrit « Ne pas toucher », c'est que ce code ne te concerne pas.

2 Point de départ des personnages

Les personnages doivent apparaître à des endroits aléatoires et différents. En d'autres termes, il ne peut y avoir qu'un seul personnage par endroit. La liste des ces points d'apparition (`SpawnPoints`) est disponible dans le `GameController` (voir la propriété `CityObjects`).



3 Senseurs des personnages



Les personnages possèdent à la fois un *collider* et un *trigger*. Le *trigger* sert à détecter les objets dans l'environnement. Crée une classe `CharacterSensor` dans le but de détecter les objets suivants :

- **Déchets** : Si le personnage est *propre*, placer le déchet détecté dans le *blackboard* comme étant le dernier déchet aperçu. Lorsque ce déchet n'est plus détecté, enlève-le du *blackboard*.
- **Amis** : Si le personnage détecte un de ses amis (voir le *blackboard*), place-le dans le *blackboard* comme étant le dernier ami aperçu. Encore une fois, lorsqu'il n'est plus détecté, enlève-le du *blackboard*.

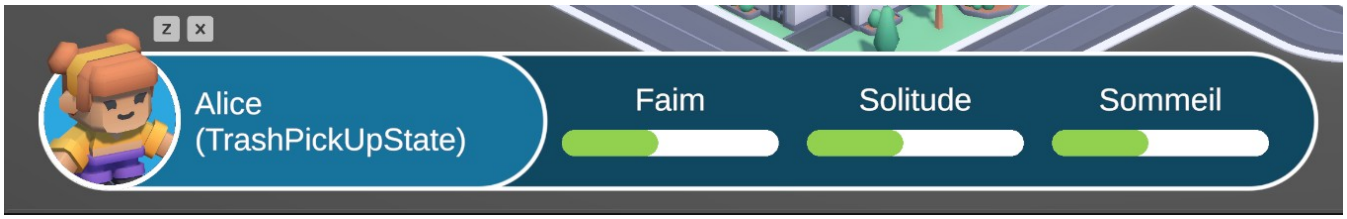
Important

Les *triggers* peuvent détecter d'autres *triggers*. Il est donc possible que ton senseur ait détecté le senseur d'un autre personnage au lieu d'avoir détecté son *collider*. Vérifie donc que c'est bel et bien un *trigger* avec la propriété `isTrigger` avant de l'ajouter au *blackboard*.

Les déchets quant à eux n'ont pas de *collider*, juste un *trigger*. C'est pour éviter que les personnages entrent en collision avec lorsqu'ils se déplacent. C'est à prendre en considération dans ton code.

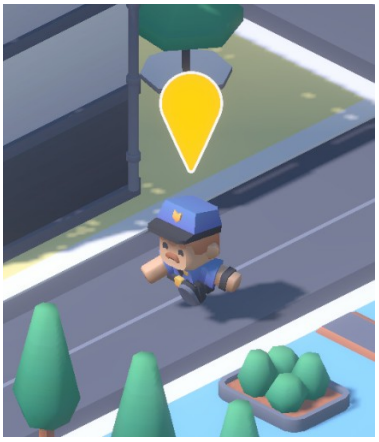
4 Interface (HUD) du jeu

En bas à gauche de l'écran se trouve une interface montrant des informations sur un personnage précis. Pour rappel, tu trouveras dans la classe `GameController` une liste de tous les personnages du jeu.



Tu dois afficher dans cette interface :

- Le nom du personnage (voir `Character.FullName`).
- Son avatar (voir `Character.Avatar`)
- Le nom de l'état actuel (voir `CharacterStateMachine.CurrentStateName`).
- L'état des trois signes vitaux (voir `CharacterVitals`).

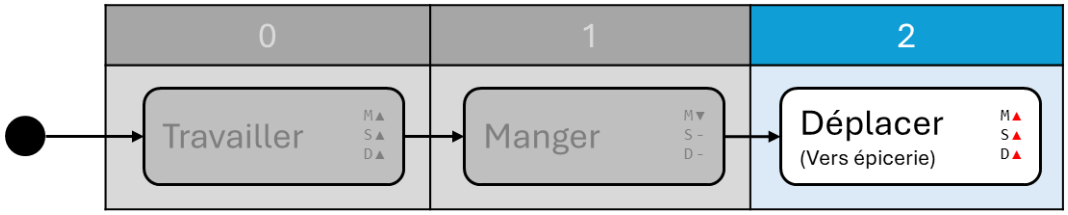


Les barres montrant les signes vitaux sont des `ScrollBar`. Elles possèdent un attribut nommé `size`, dont la valeur est entre `0` et `1`. En guise de bonus, tu peux changer la couleur de la barre selon l'état des signes vitaux : vert si ça va très bien, jaune quand c'est normal et rouge quand c'est critique.

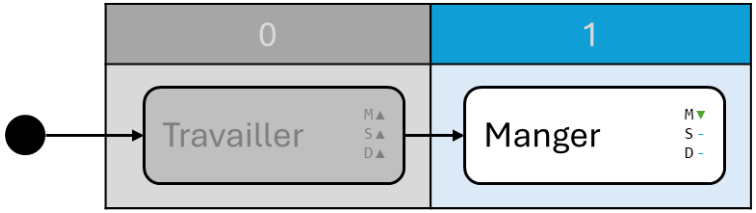
Tous les personnages possèdent un `GameObject` nommé `Pointer`, visant à montrer la position du personnage sélectionné. Les touches Z et X doivent permettre de passer d'un personnage à un autre. Tu dois activer le pointeur du personnage actuel et désactiver les autres.

5 Machine à état des personnages

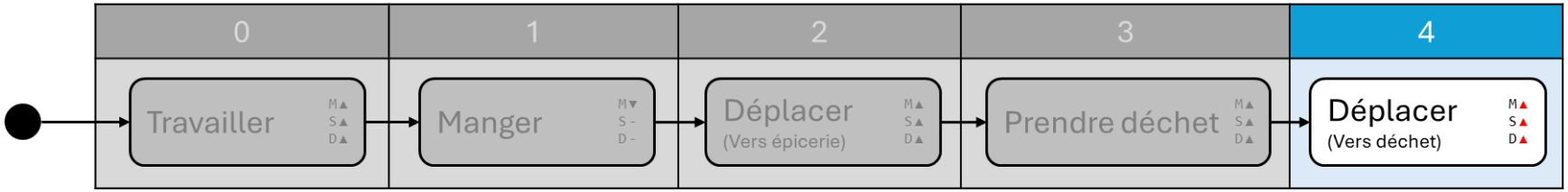
Tu dois implémenter une machine à états utilisant une pile. L'état sur le dessus de la pile est l'état actif. Dans ce type de machine, chaque état représente une tâche à accomplir. Certaines tâches ont des prérequis, ce qui conduit à empiler d'autres tâches pour satisfaire ces prérequis. Par exemple, pour manger, le personnage doit se déplacer en premier lieu dans une épicerie.



Lorsqu'une tâche est complétée, elle est retirée du dessus de la pile. Ainsi, la tâche du dessous peut reprendre son cours.



Ce type de machine à état est idéal pour les situations où un acteur doit interrompre sa tâche actuelle pour en réaliser une plus urgente. Lorsque la tâche plus urgente est complétée, il peut alors revenir à ce qu'il faisait auparavant. Dans l'exemple ci-dessous, le personnage voulait manger, mais durant son déplacement, il a été interrompu par la présence d'un déchet à ramasser, déchet vers lequel se déplace.



Tous les personnages commencent dans l'état « Travailler ». Pour éviter de vider complètement la pile, cette tâche ne peut pas être complétée. Ainsi, les personnages ont toujours quelque chose à faire. Dans d'autres jeux, il s'agirait de l'état Neutre (ou *Idle* en anglais).

Le tableau ci-dessous présente chaque état, leurs transitions et comments ils impactent les signes vitaux. Trois impacts sont possibles : ▲ augmentation, ▼ diminution ou — neutre.

État	Transitions	Impact sur les signes vitaux		
		Manger	*Socialiser*	*Dormir*
Travailler	• Déplacer (Vers travail) – Si trop loin du lieu de travail. • Manger – Si faim est trop élevée. • Socialiser – Si solitude est trop élevée. • Dormir – Si sommeil est trop élevé. *Note 1 : Lorsque le personnage est à son lieu de travail, simule qu'il vient d'y entrer le rendant invisible. Lorsque le personnage arrête de travailler, rends-le visible afin de simuler qu'il vient d'en sortir. Consultez les méthodes disponibles dans `Character`.*	▲	▲	▲
Déplacer (Vers destination)	• Prendre déchet – Si personnage « propre » voit un déchet. • Lancer déchet – Si personnage « malpropre » doit lancer un déchet. • Saluer ami – Si personnage aperçoit un ami. *Note 1 : La propriété `ShouldThrowTrash` dans `Character` devient `true` pour indiquer que le personnage doit lancer un déchet. Sa gestion est automatique.* *Note 2 : Cet état ne peut transitionner vers un autre état que si la destination est un bâtiment. Cette restriction vise à empêcher qu'un déplacement vers un déchet soit interrompu par la détection d'un autre déchet en cours de route, ce qui augmenterait indument la taille de la pile.*	▲	▲	▲
Manger	• Déplacer (Vers épicerie) – Si trop loin de l'épicerie choisie. *Note 1 : L'épicerie de destination doit être choisie aléatoirement.* *Note 2 : Tout comme le lieu de travail, rends le personnage invisible pour simuler son entrée dans l'épicerie et rends le visible pour l'en faire sortir.*	▼	—	—
Socialiser	• Déplacer (Vers café) – Si trop loin du café choisi. *Note : Le café doit être choisi aléatoirement lorsque le personnage débute cette tâche.*	—	▼	—
Dormir	• Déplacer (Vers maison) – Si trop loin de la maison du personnage. *Note : Un personnage possède une seule maison (voir son `CharacterBlackboard`).*	—	—	▼

État	Transitions	Impact sur les signes vitaux		
		Manger	*Socialiser*	*Dormir*
Prendre déchet	• Déplacer (Vers déchet) – Si trop loin du déchet aperçu. *Note 1 : Le déchet pourrait être ramassé par un autre personnage. Il est donc nécessaire de vérifier s'il est toujours disponible avant de poursuivre dans cet état. Sinon, annule la tâche.* *Note 2 : L'action de prendre un déchet nécessite de s'y déplacer en premier lieu. Ensuite, tu dois déclencher une animation et attendre la fin de l'animation avant de sortir de l'état (voir méthodes `PickupTrash` et `IsPickingTrash` dans `Character`).*	▲	▲	▲
Lancer déchet	• *Aucune transition.* *Note : Dans cet état, tu dois déclencher une animation et attendre la fin de cette animation avant d'en sortir (voir méthodes `ThrowTrash` et `IsThrowingTrash`).*	▲	▲	▲
Saluer ami	• *Aucune transition.* *Note : Encore une fois, tu dois déclencher l'animation de salutation et en attendre la fin avant de sortir de cet état (voir méthodes `GreetCharacter` et `IsGreetingCharacter`).*	—	▼	—

Voici d'autres informations importantes sur la machine à état ainsi que quelques conseils :

- La machine à état doit être gérée dans une classe `CharacterStateMachine`. Tu auras à la programmer au complet.
- Pour simplifier, tu n'as pas à recycler les états ou à limiter le travail du *Garbage Collector* pour la machine à état.
- Tous tes états doivent avoir une méthode `Enter`, `Update` et `Exit`, tel que montré en classe. L'appel de ces méthodes doit se faire au bon moment du cycle de vie de la machine à état (encore une fois, tel que montré en classe).
- Certains états devraient recevoir des paramètres à leur construction. Par exemple, l'état « Déplacer » devrait recevoir la destination à atteindre et l'état « Prendre déchet » devrait recevoir le déchet à ramasser.
- N'oublie pas de compléter l'interface (le UI) en affichant le nom de l'état actuel du personnage.
- En cas de questionnement (ou si tu es confus), contacte rapidement ton professeur.

6 Éléments Bonus

Voici une liste d'autres éléments bonus que vous pouvez réaliser si le cœur vous en dit :

- Effectuer le recyclage des états (très complexe, à faire que si tu as terminé le reste).
- Ajouter une musique qui boucle correctement (ce qui est plus facile à dire qu'à faire).
- Faire apparaitre (ou disparaitre) les déchets au bon moment durant les animations.
- Créer une caméra différente (où l'on voit mieux l'action, à discuter avec le professeur).
- Changer la couleur des barres montrant les signes vitaux selon la valeur (vert, jaune, rouge).

Contraintes

- Tout le texte doit être en **français**. La qualité du français sera corrigée.
- Le code doit être en **anglais**. Les commentaires peuvent être en français si souhaité.
- L'utilisation du *New Input System* de Unity est obligatoire.
- Tu dois avoir un `Finder` (ce sera très difficile sans lui de toute façon).
- Il est **strictement interdit** d'ajouter des scripts dans le dossier `Util`.

Plagiat et fraude

Toute forme de fraude, plagiat ou tricherie sera sévèrement sanctionnée. Cela inclut notamment le fait de s'approprier le travail d'autrui, y compris celui généré par une intelligence artificielle. Le simple fait d'obtenir et même de partager des réponses est automatiquement considéré comme du plagiat.

Tout comportement de ce type entraînera automatiquement l'attribution de la note de 0 %. La fraude sera aussi déclarée à la direction des études, qui pourra imposer d'autres sanctions au besoin, tel que l'expulsion permanente du programme ou du collège.

Modalités de remise

Remets ton projet sur LÉA à l'intérieur d'une archive *Zip*. Une pénalité de 15 % est appliquée en cas de retard de moins d'une journée. Au-delà de ce délai, la note de 0 % est attribuée. Supprime les fichiers et dossiers superflus (notamment `.vs`, `.idea`, `.git`, Library, Logs, obj, Temp, UserSettings, ainsi que les fichiers `.sln`, `.csproj`, .user et cet énoncé).

Évaluation

1 Livrables

Critères	Pondération
Utilisation correcte du C# et des scripts Code clair et algorithmes compréhensibles (documentées si nécessaire). Respect de l'encapsulation. Respects des standards du langage. Code de qualité. Exploitation judicieuse des possibilités du langage (ne pas recoder ce qui existe déjà). Faire des petits scripts reliés à un comportement donné. Bonnes solutions pour la communication entre scripts. (Peut être ajusté si pas suffisamment de contenu).	20 %
Utilisation correcte de Unity et de ses concepts. Utilisation correct et judicieuse des *Game Objects* et des composants. Respect de la boucle de jeu Unity et de ses parties. Utilisation judicieuse des *colliders*, *triggers* et de la physique. Communication entre *Game Objects*. Recyclage d'objets et stabilisation de la mémoire. Gestion des entrants (inputs). Utilisation judicieuse du moteur Unity (*Project*, *Hierarchy* et *Inspector*). (Peut être ajusté si pas suffisamment de contenu).	30 %
Interface (UI) Interface montrant l'état des personnages. Possible de changer de personnage. Le pointeur change aussi pour montrer sa position.	10 %
Machine à état. Machine à état implémenté avec une pile. Comprends tous les comportements requis (travailler, se déplacer, manger, socialiser, dormir, ramasser un déchet, lancer un déchet et saluer un ami). Les décisions sont prises dans la machine à états.	40 %

2 Pénalités

Pénalités – Rigueur
Qualité générale du code (incluant, mais sans s'y limiter) : • Propreté générale du code. • Mauvaises pratiques de programmation. • Formatage ou indentation déficiente. • Standard de nommage non respecté. • Cohérence de l'ensemble de l'œuvre. • Orthographe et grammaire (0.5% jusqu'à concurrence de 20 %).
Remise du travail (incluant, mais sans s'y limiter) : • Erreur ou avertissement à l'interprétation du code. • Non-respect des consignes de remise. • Fichier inutile ou temporaire remis avec le projet. • Travail remis en retard.